

Trabajo Práctico Integrador – Metodologías de Sistemas

II Entrega final: repositorio Git + informe + defensa

Fecha:

21/05/2026

Integrantes:

- **Hoffman, Diana**
- **Krumrick, Silvia**
- **Mardon, Florencia**
- **Perrón, Graciela**
- **Sextua, Sandra**

Estructura por semana:

Semana 1 – Refactorización Inicial y Calidad de Código

Objetivo de la semana

Durante la primera semana se realizó una revisión del código base del sistema de feria de emprendedores. El objetivo principal fue identificar problemas de diseño y aplicar una refactorización inicial, manteniendo la misma funcionalidad externa del sistema.

Esta etapa estuvo enfocada en mejorar la claridad del código, reducir duplicaciones, separar responsabilidades, proteger el estado interno de los objetos y aplicar principios básicos de código limpio y SOLID.

1. Code smells detectados

1.1. Nombres poco descriptivos

En la clase Emprendedor se utilizaban atributos con nombres de un solo carácter, como n, t, m, cat y prods. Estos nombres dificultaban la lectura del código y hacían menos clara la intención de cada atributo.

Para corregir este problema, se reemplazaron por nombres descriptivos: nombre, telefono, email, categoria y productos.

1.2. Atributos públicos y falta de encapsulamiento

Se detectó que varias clases exponían directamente sus atributos como public. Esto permitía modificar el estado interno de los objetos desde cualquier parte del sistema, especialmente desde Main y GestorFeria.

Esta situación generaba un diseño poco seguro y difícil de mantener, ya que no existía control sobre cómo se modificaban los datos internos de cada objeto.

1.3. Mezcla de responsabilidades

El método mostrarInfoYValidar() de la clase Emprendedor realizaba dos tareas diferentes: generar información para mostrar y ejecutar reglas de validación de negocio.

Esta mezcla de responsabilidades afectaba la cohesión de la clase y dificultaba la modificación o reutilización del código.

1.4. Código duplicado

En la clase Producto existían dos métodos con lógica similar para determinar si el stock era bajo: hayStockBajo() e isStockBajo().

También se encontró duplicación en la clase Reportes, donde se calculaba de forma redundante el acumulado de ventas totales.

1.5. Clase con demasiadas responsabilidades

La clase GestorFeria concentraba demasiadas tareas: registro de emprendedores, registro de productos, validaciones de stock, cobros, mutación de colecciones y coordinación general del sistema.

Esto generaba un alto acoplamiento y hacía que la clase fuera más difícil

2. Refactorizaciones aplicadas

2.1. Renombrado de atributos

Se reemplazaron los nombres poco claros por nombres descriptivos, mejorando la legibilidad del código y facilitando su comprensión.

- ❖ n → nombre
- ❖ t → telefono
- ❖ m → email
- ❖ cat → categoria
- ❖ prods → productos

2.2. Encapsulamiento de atributos

Los atributos principales de las clases fueron modificados de public a private. De esta forma, el estado interno de los objetos quedó protegido frente a modificaciones directas desde otras clases.

Para acceder a la información o modificarla de manera controlada, se agregaron getters y métodos específicos según la responsabilidad de cada clase.

2.3. Separación de responsabilidades

Se separó la generación de información de la validación de datos. El método mostrarInfoYValidar() fue reemplazado por una estructura más clara, donde la salida de datos queda separada de las reglas de validación.

La generación de información quedó a cargo de un método específico, mientras que las validaciones fueron delegadas a la clase Validadores.

2.4. Centralización de validaciones

Se reutilizó la clase Validadores para concentrar validaciones de email, teléfono, categoría, precio y stock.

Esto permitió evitar reglas dispersas en distintas clases y dejó el sistema más preparado para incorporar nuevas validaciones en el futuro con menor impacto sobre el resto del código.

2.5. Eliminación de lógica duplicada

En la clase Producto se eliminó la duplicación relacionada con la consulta de stock bajo. La lógica quedó unificada en un único método llamado tieneStockBajo().

En la clase Reportes se reutilizó el método calcularVentasTotales() dentro del resumen ejecutivo, evitando repetir el mismo cálculo en distintos lugares.

2.6. Protección de listas internas

Se evitó el acceso directo a las listas internas desde Main y Reportes. Para ello, se agregaron métodos públicos en GestorFeria que permiten registrar emprendedores, registrar productos, registrar ventas y consultar información sin exponer directamente la estructura interna de datos.

Con esta modificación, las colecciones internas quedan protegidas y el sistema mantiene un mayor control sobre las operaciones que se realizan sobre ellas.

3. Principios SOLID aplicados

3.1. Principio de Responsabilidad Única (SRP)

Se aplicó el principio de responsabilidad única al separar mejor las funciones de cada clase.

- ❖ Emprendedor representa los datos y comportamientos propios de un emprendedor.
- ❖ Producto representa los datos y comportamientos propios de un producto.
- ❖ Validadores concentra las reglas de validación.
- ❖ GestorFeria coordina las operaciones generales de la feria.
- ❖ Reportes se encarga de generar información resumida del sistema.

Gracias a esta separación, cada clase queda más enfocada en una responsabilidad concreta, lo que mejora la claridad y mantenibilidad del sistema.

3.2. Principio de Abierto/Cerrado (OCP)

La refactorización dejó el código más preparado para extender funcionalidades sin modificar directamente toda la lógica existente.

Por ejemplo, al centralizar validaciones y separar responsabilidades, futuras reglas de negocio podrán agregarse con menor impacto sobre las clases principales.

3.3. Encapsulamiento como mejora de diseño

Aunque el encapsulamiento no es un principio SOLID en sí mismo, fue una mejora importante aplicada durante esta etapa.

Al proteger los atributos y listas internas, se evita que otras clases modifiquen directamente el estado de los objetos. Esto permite mantener mayor control sobre los datos y reduce errores derivados de accesos indebidos.

Semana 2 – Patrón de Diseño

4. Problema identificado

Durante la revisión arquitectónica del sistema, se detectó un problema estructural centralizado en la clase GestorFeria. A pesar de las refactorizaciones iniciales, esta clase continuaba acumulando múltiples responsabilidades de negocio, funcionando como una clase con demasiadas funciones dentro del sistema.

GestorFeria se encargaba de la validación y registro de emprendedores, la administración de productos, el control de inventario y stock, el registro de ventas y el procesamiento de cobros. Esta concentración de responsabilidades aumentaba el acoplamiento entre módulos y dificultaba el mantenimiento del código, ya que cualquier cambio en una regla de negocio podía impactar directamente en una clase central del sistema.

Además, el cliente del sistema, representado por la clase Main, dependía de una clase demasiado cargada de lógica interna. Esto reducía la claridad del diseño y hacía más difícil extender el sistema sin afectar funcionalidades ya existentes.

5. Patrón de diseño implementado

Para resolver esta deficiencia arquitectónica, se optó por implementar el patrón de diseño estructural Facade (Fachada).

Se procedió a dividir y encapsular la lógica interna de la feria creando un subsistema compuesto por tres clases especialistas:

- ❖ GestorEmprendedores: Administra la colección de usuarios y sus validaciones.
- ❖ GestorInventario: Centraliza la creación, búsqueda y modificación de stock de artículos.
- ❖ GestorVentas: Controla el registro de transacciones comerciales y procesa los pagos.

A partir de esta separación, la clase GestorFeria fue rediseñada para actuar como una fachada del sistema. En lugar de concentrar directamente toda la lógica de negocio, mantiene referencias a los gestores internos y delega cada operación al componente correspondiente.

De esta forma, Main continúa utilizando una interfaz simple para operar con el sistema, sin necesidad de conocer en detalle cómo se organizan internamente los distintos gestores.

La alternativa simple habría sido mantener toda la lógica dentro de GestorFeria y continuar agregando métodos a esa misma clase a medida que el sistema creciera. Si bien esta solución era más rápida de implementar, no era adecuada a largo plazo porque aumentaba el acoplamiento, reducía la cohesión y mantenía el problema de la clase con demasiadas responsabilidades.

Con la implementación de Facade, se conserva un punto de acceso sencillo para el cliente, pero se distribuye la lógica en clases internas más específicas.

6. Mejora obtenida.

La implementación de Facade permitió reducir la concentración de responsabilidades en GestorFeria y mejorar la organización general del sistema. La clase principal deja de actuar como una clase cargada de lógica de negocio y pasa a funcionar como una interfaz de acceso simplificada hacia los distintos subsistemas.

Esta mejora favorece un mayor cumplimiento del Principio de Responsabilidad Única, ya que cada gestor interno se ocupa de una parte concreta del sistema. También reduce el acoplamiento entre el cliente y las clases internas, porque Main no necesita conocer directamente cómo se registran emprendedores, cómo se administra el inventario o cómo se procesan las ventas.

Además, el diseño queda mejor preparado para futuras extensiones. Por ejemplo, si más adelante se decide modificar la forma de almacenar productos, incorporar una base de datos o cambiar la lógica de stock, esos cambios podrían concentrarse principalmente en GestorInventario, sin alterar el uso externo del sistema.

7. Comparación antes/después

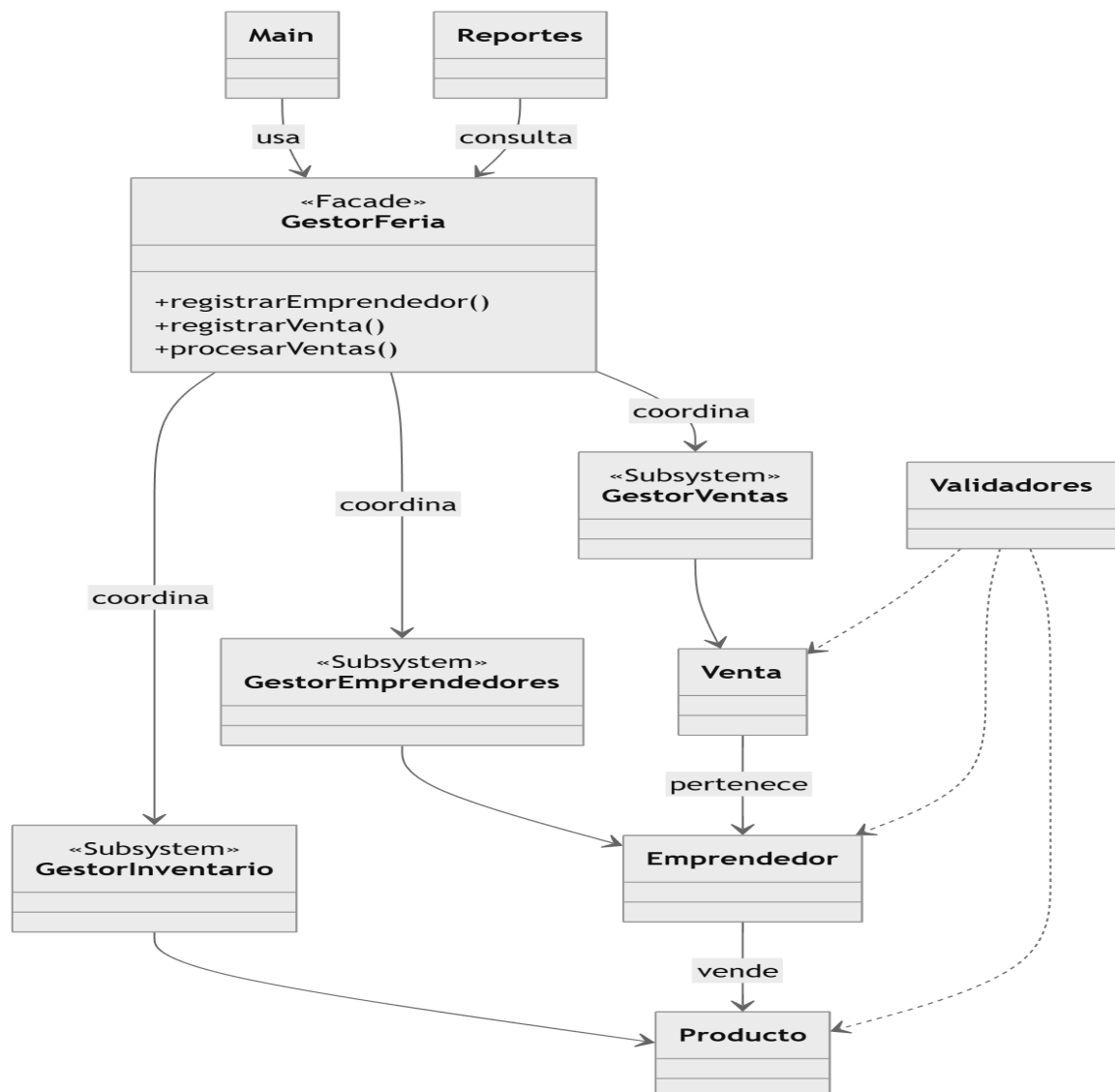
Antes de aplicar Facade:

Main dependía de una clase GestorFeria con múltiples responsabilidades concentradas. La misma clase registraba emprendedores, administraba productos, controlaba stock, registraba ventas y procesaba cobros.

Después de aplicar Facade:

Main interactúa con GestorFeria como fachada. GestorFeria mantiene una interfaz simple, pero delega internamente las operaciones en gestores especializados: GestorEmprendedores, GestorInventario y GestorVentas.

Diagrama UML aplicando Patrón Facade.



Conclusión

El patrón Facade resultó apropiado porque permitió mantener una interfaz sencilla para el uso del sistema, pero ocultando la complejidad interna detrás de una clase fachada. La mejora principal fue la reducción del acoplamiento, la separación de responsabilidades y una arquitectura más clara y mantenible.

SEMANA 3 – TESTING AUTOMATIZADO

Objetivo de la semana

Implementar pruebas automatizadas sobre las funcionalidades principales del sistema de gestión de la Feria de Emprendedores, utilizando JUnit y aplicando conceptos de TDD (Test Driven Development).

Herramientas utilizadas

- Java 17
- Maven
- JUnit 5
- Mockito

Pruebas implementadas

1. ProductoTest
 - Verificación de detección de stock bajo.
 - Verificación de control de stock insuficiente.
 - Verificación de productos premium mediante la nueva funcionalidad desarrollada con TDD.
2. VentaTest
 - Verificación del cálculo correcto del subtotal.
 - Verificación de la aplicación de descuentos por cantidad.
3. ValidadoresTest
 - Validación de correo electrónico correcto.
 - Validación de correo electrónico incorrecto.
 - Validación de categorías permitidas.

Mocks

Se incorporó la dependencia Mockito al proyecto para permitir la simulación de objetos durante las pruebas y preparar la arquitectura para futuros escenarios de aislamiento de dependencias.

Aplicación de TDD

Se desarrolló una nueva funcionalidad denominada `esPremium()` dentro de la clase `Producto`.

- Fase Roja:
Se creó inicialmente una prueba que verificaba si un producto con precio superior a \$10.000 era considerado premium.
- Fase Verde:
Se implementó el método `esPremium()` para satisfacer la prueba.
- Fase Refactor:
Se mantuvo una implementación simple y clara respetando los principios de diseño aplicados en etapas anteriores.

Resultados obtenidos

La ejecución de la suite de pruebas produjo los siguientes resultados:

Tests run: 9

Failures: 0

Errors: 0

Skipped: 0

Resultado final: BUILD SUCCESS

Conclusión

La incorporación de las pruebas automatizadas nos permitió verificar el correcto funcionamiento de los componentes críticos del sistema y a su vez, sentó las bases para la integración continua en la siguiente etapa del proyecto (sem 4).

SEMANA 4 – INTEGRACIÓN CONTINUA

Se configuró un pipeline de Integración Continua utilizando GitHub Actions.

Se creó el archivo: `.github/workflows/ci.yml`

El workflow fue configurado para ejecutarse automáticamente ante cada push o pull request realizado sobre la rama principal del proyecto (main).

Las tareas automatizadas definidas fueron:

- Descarga del código fuente.
- Configuración del entorno Java 17.
- Ejecución automática de las pruebas unitarias mediante Maven.

Previamente se verificó la correcta ejecución local de la suite de pruebas obteniendo:

Tests run: 9

Failures: 0

Errors: 0

BUILD SUCCESS

Durante la validación del workflow, GitHub Actions reconoció correctamente el archivo de configuración y el job definido. Sin embargo, cabe aclarar que la ejecución automática quedó bloqueada por una restricción administrativa de la cuenta de GitHub asociada al repositorio (“account locked due to a billing issue”), situación ajena al desarrollo del proyecto y a los participantes de esta entrega con licencias gratuitas.

La configuración del pipeline quedó implementada y lista para su ejecución una vez resuelta dicha restricción.

